LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – Rekrusif

Serlin Aprilia

109082530032

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Rekursif merupakan salah satu metode dalam pemrograman dimana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan. Konsep ini sangat berguna untuk menyederhanakan permasalahan yang dapat dipecah menjadi sub-masalah yang lebih kecil dengan pola yang sama. Dalam rekursi terdapat dua komponen utama, yaitu base case (kondisi dasar) dan recursive case (pemanggilan ulang fungsi). Base case berfungsi untuk menghentikan proses rekursi agar tidak terjadi pemanggilan fungsi secara tak terbatas, sedangkan recursive case merupakan bagian dimana fungsi memanggil dirinya sendiri dengan parameter yang lebih sederhana atau mendekati kondisi dasar.

Pada implementasinya dalam bahasa Go, fungsi rekursif bekerja dengan memanfaatkan stack memori, dimana setiap pemanggilan fungsi akan disimpan hingga mencapai base case, kemudian hasilnya dikembalikan secara berurutan. Contoh penerapan rekursi dapat ditemukan pada berbagai permasalahan seperti perhitungan faktorial, deret Fibonacci, pencarian faktor bilangan, serta penampilan pola bilangan seperti bilangan ganjil.

Penggunaan rekursi seringkali membuat kode menjadi lebih sederhana dan mudah dipahami dibandingkan dengan penggunaan perulangan (iterasi), terutama untuk permasalahan yang memiliki pola berulang. Namun, rekursi juga memiliki kelemahan seperti penggunaan memori yang lebih besar dan potensi terjadinya stack overflow jika tidak dirancang dengan benar. Oleh karena itu, penting untuk memastikan bahwa setiap fungsi rekursif memiliki kondisi dasar yang jelas dan proses yang menuju ke kondisi tersebut.

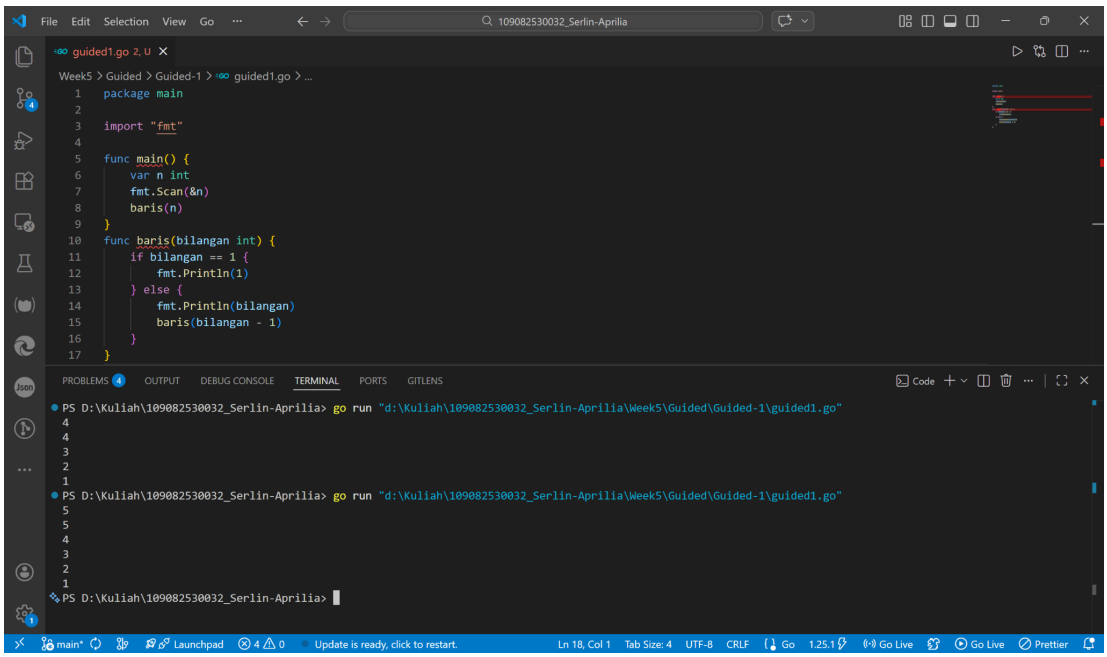
B. Guided

1. Membuat baris bilangan dari n hingga 1

```
package main


import "fmt"


func main() {
     var n int
```

```
        fmt.Scan(&n)
        baris(n)
}
func baris(bilangan int) {
        if bilangan == 1 {
                fmt.Println(1)
        } else {
                fmt.Println(bilangan)
                baris(bilangan - 1)
        }
}
```

Output :



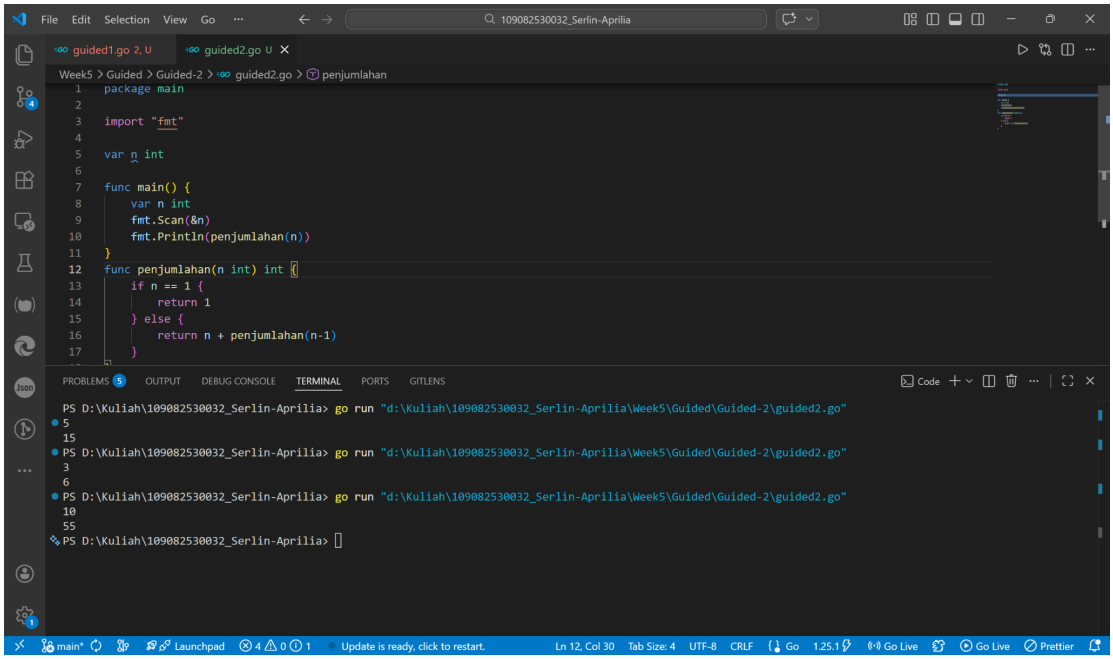
Program Go tersebut menggunakan fungsi rekursif untuk menampilkan deret angka dari nilai yang dimasukkan pengguna hingga angka 1 secara menurun. Pada fungsi main, program mendeklarasikan variabel n bertipe integer lalu membaca input pengguna menggunakan fmt.Scan(&n), kemudian memanggil fungsi baris(n). Fungsi baris menerima parameter bilangan dan mengecek apakah nilainya sama dengan 1 sebagai base case; jika iya, maka program mencetak angka 1 dan berhenti. Jika tidak, program

mencetak nilai bilangan terlebih dahulu lalu memanggil kembali fungsi baris(bilangan - 1) sehingga terjadi pemanggilan berulang (rekursif) dengan nilai yang terus berkurang sampai mencapai 1. Oleh karena itu, jika pengguna memasukkan angka 5, output yang dihasilkan adalah 5, 4, 3, 2, 1 secara berurutan ke bawah.

2. Menghitung hasil penjumlahan 1 hingga n

```
package main

import "fmt"

var n int

func main() {
      var n int
      fmt.Scan(&n)
      fmt.Println(penjumlahan(n))
}
func penjumlahan(n int) int {
      if n == 1 {
            return 1
      } else {
            return n + penjumlahan(n-1)
      }
}
```

Output :

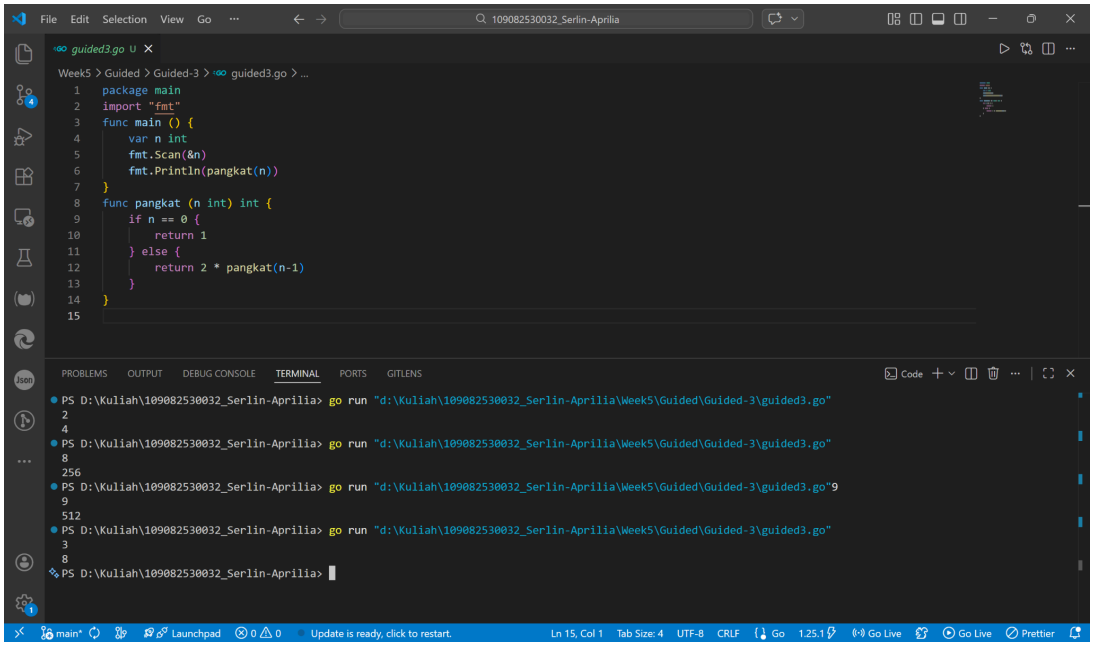


Program Go tersebut menggunakan fungsi rekursif untuk menghitung jumlah deret bilangan dari 1 sampai dengan nilai yang dimasukkan pengguna. Pada bagian main, program membaca input angka n menggunakan fmt.Scan(&n) lalu memanggil fungsi penjumlahan(n) dan mencetak hasilnya dengan fmt.Println(). Fungsi penjumlahan(n int) int merupakan fungsi rekursif yang memiliki base case saat n == 1, dimana fungsi langsung mengembalikan nilai 1. Jika n lebih dari 1, maka fungsi akan mengembalikan hasil n + penjumlahan(n-1) sehingga terjadi pemanggilan fungsi berulang dengan nilai yang terus berkurang sampai mencapai 1. Misalnya jika input 5, maka perhitungannya menjadi 5 + 4 + 3 + 2 + 1 sehingga menghasilkan output 15, seperti yang terlihat pada hasil terminal.

3. Mencari dua pangkat n atau 2 pangkat n

```
package main
import "fmt"
func main () {
      var n int
      fmt.Scan(&n)
      fmt.Println(pangkat(n))
}
func pangkat (n int) int {
      if n == 0 {
            return 1
      } else {
            return 2 * pangkat(n-1)
      }
}
```

Output :



Program Go tersebut digunakan untuk menghitung perpangkatan bilangan 2 menggunakan fungsi rekursif. Pada fungsi main, program mendeklarasikan variabel n bertipe integer lalu membaca input dari pengguna menggunakan fmt.Scan(&n). Setelah itu, program memanggil fungsi pangkat(n) dan mencetak hasilnya dengan fmt.Println(). Fungsi pangkat(n int) int memiliki base case ketika n == 0, dimana fungsi mengembalikan nilai 1 karena setiap bilangan berpangkat 0 hasilnya adalah 1. Jika n tidak sama dengan 0, maka fungsi akan mengembalikan 2 * pangkat(n-1) sehingga terjadi pemanggilan fungsi secara berulang dengan nilai n yang terus berkurang hingga mencapai 0. Misalnya jika input 3, maka prosesnya menjadi 2 × 2 × 2 = 8, sedangkan jika input 8 maka hasilnya 2^8 = 256, sesuai dengan output yang terlihat pada terminal.

C. Unguided

1. Soal Unguided 1

1) Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

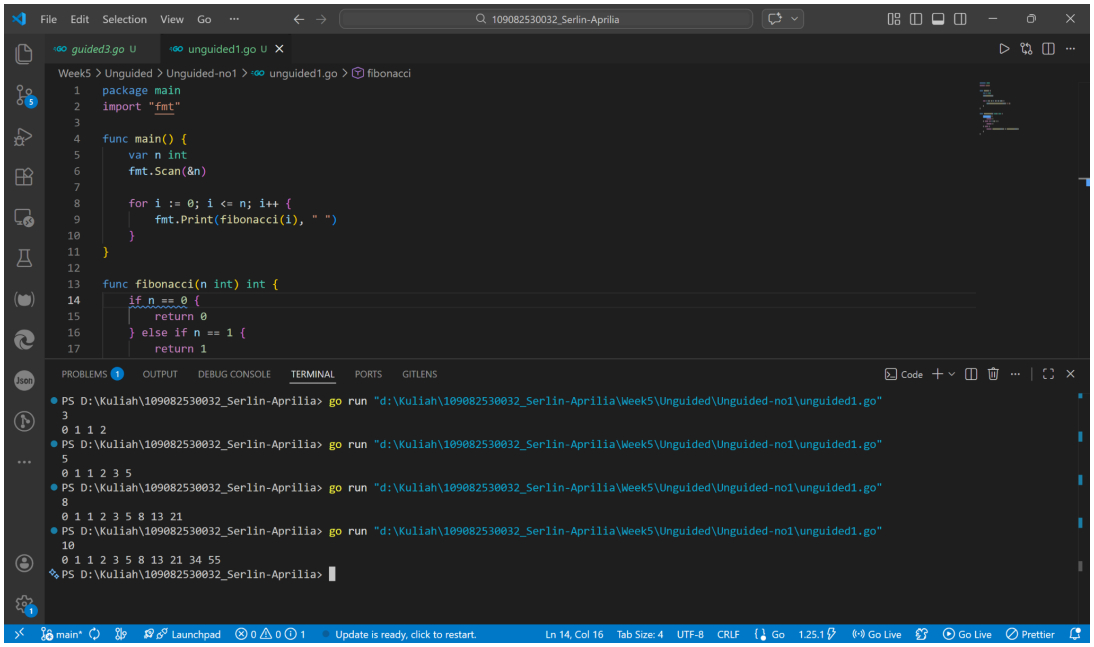
n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban :

```
package main
import "fmt"

func main() {
	var n int
	fmt.Scan(&n)

	for i := 0; i <= n; i++ {
		fmt.Print(fibonacci(i), " ")
	}
}

func fibonacci(n int) int {
	if n == 0 {
		return 0
	} else if n == 1 {
		return 1
	} else {
		return fibonacci(n-1) + fibonacci(n-2)
	}
```

```
}
```

Output :



Penjelasan :

Program Go tersebut digunakan untuk menampilkan deret Fibonacci menggunakan fungsi rekursif. Pada fungsi main, program mendeklarasikan variabel n bertipe integer lalu membaca input dari pengguna menggunakan fmt.Scan(&n). Setelah itu, program menggunakan perulangan for dari i = 0 sampai i <= n untuk mencetak setiap nilai deret Fibonacci dengan memanggil fungsi fibonacci(i). Fungsi fibonacci(n int) int merupakan fungsi rekursif yang memiliki base case yaitu ketika n == 0 mengembalikan nilai 0 dan ketika n == 1 mengembalikan nilai 1. Jika nilai n lebih dari 1, maka fungsi akan memanggil dirinya sendiri menggunakan rumus fibonacci(n-1) + fibonacci(n-2) sehingga setiap nilai merupakan hasil penjumlahan dua suku sebelumnya. Karena itu, jika pengguna memasukkan angka 10, maka output yang dihasilkan adalah 0 1 1 2 3 5 8 13 21 34 55, sesuai dengan hasil yang terlihat pada terminal.

2. Soal Unguided 2 nomer 3

3) Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 5

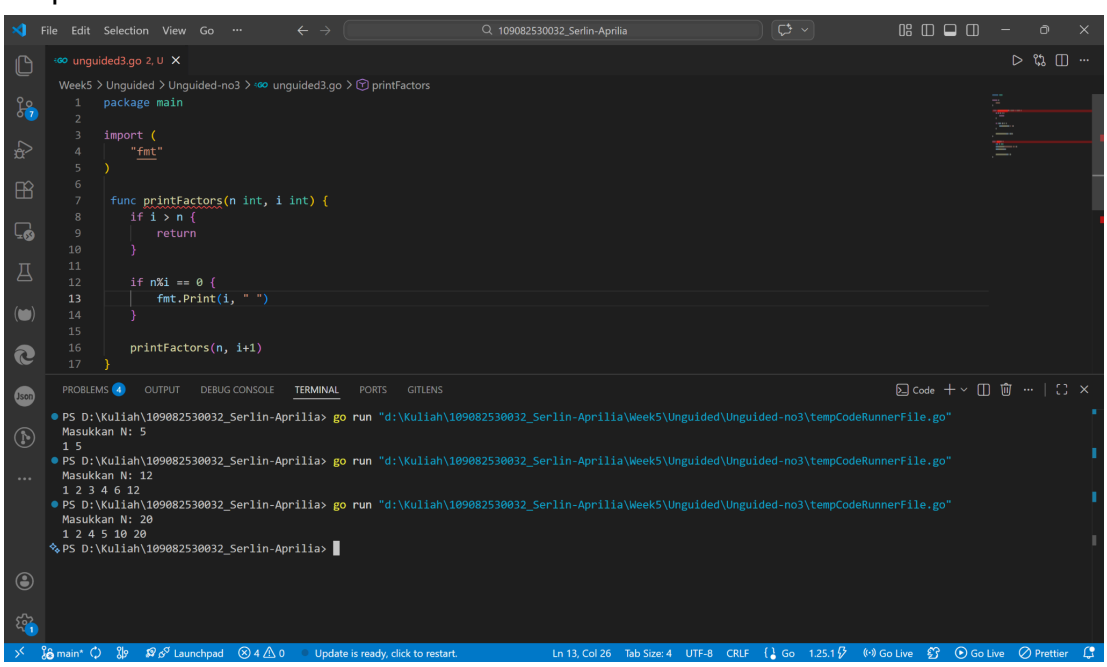
2	12	1 2 3 4 6 12

Jawaban :

```
package main


import (
      "fmt"
)


 func printFactors(n int, i int) {
      if i > n {
            return
      }
```

```
        if n%i == 0 {
            fmt.Print(i, " ")
        }

        printFactors(n, i+1)
}

func main() {
        var n int
        fmt.Print("Masukkan N: ")
        fmt.Scan(&n)

        printFactors(n, 1)
}
```

Output :



Penjelasan :

Program tersebut menggunakan konsep rekursif melalui fungsi printFactors(n, i) untuk menentukan dan menampilkan semua faktor dari suatu bilangan n. Proses dimulai dari nilai i = 1, kemudian fungsi akan terus memanggil dirinya sendiri dengan nilai i yang bertambah satu setiap kali pemanggilan. Pada setiap langkah, program memeriksa apakah i merupakan faktor dari n dengan kondisi n % i == 0; jika terpenuhi, maka nilai i dicetak sebagai bagian dari hasil. Rekursi akan berhenti ketika i sudah melebihi n, yang menjadi kondisi dasar (base case). Dengan pendekatan ini, program dapat menelusuri seluruh kemungkinan pembagi dari 1 hingga n secara berurutan tanpa menggunakan struktur perulangan seperti for di fungsi utama, sehingga memperlihatkan bagaimana rekursi dapat menggantikan iterasi dalam penyelesaian masalah sederhana seperti pencarian faktor bilangan.

3. Soal Unguided 3 nomer 5

5) Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Contoh masukan dan keluaran:

Fakultas Informatika
School of Computing
Telkom University
informatics lab

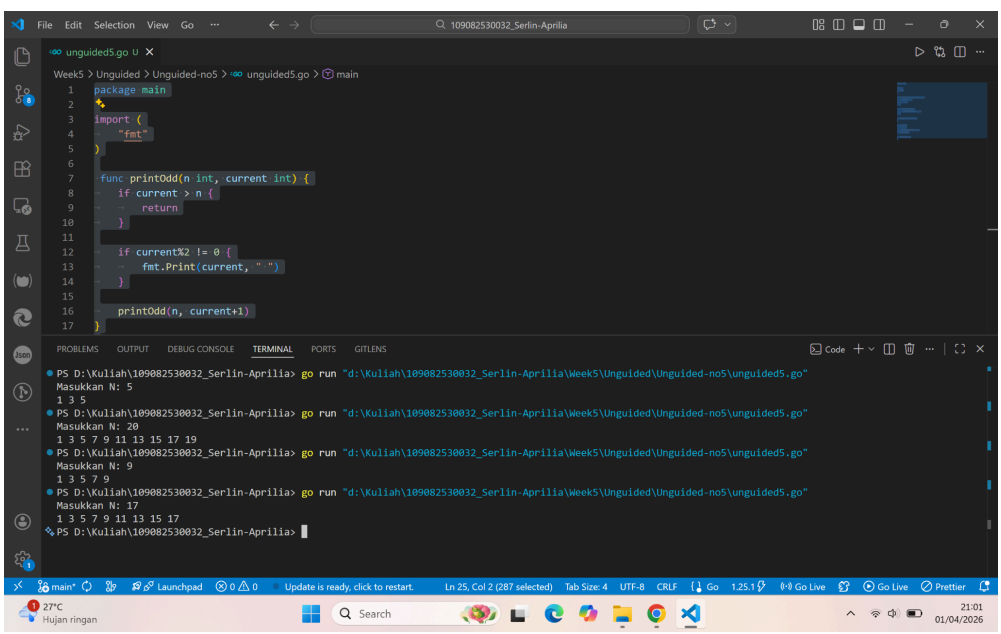
No	Masukan	Keluaran
1	5	1 3 5
2	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```
package main


import (
      "fmt"
)


 func printOdd(n int, current int) {
      if current > n {
            return
```

```go
        }

        if current%2 != 0 {
                fmt.Print(current, " ")
        }

        printOdd(n, current+1)
}

func main() {
        var n int
        fmt.Print("Masukkan N: ")
        fmt.Scan(&n)

        printOdd(n, 1)
}
```

Output :



Penjelasan :

Program tersebut menggunakan fungsi rekursif printOdd(n, current) untuk menampilkan deret bilangan ganjil dari 1 hingga N. Proses dimulai dari nilai current = 1, lalu fungsi akan memeriksa setiap angka apakah ganjil dengan kondisi current % 2 != 0; jika iya, maka angka tersebut dicetak. Setelah itu, fungsi memanggil dirinya sendiri dengan current+1 untuk melanjutkan ke angka berikutnya. Rekursi akan berhenti ketika current lebih besar dari n, yang menjadi kondisi dasar (base case). Dengan cara ini, program menelusuri semua angka dari 1 sampai N secara berurutan tanpa menggunakan perulangan eksplisit, dan hanya mencetak angka-angka ganjil sehingga menghasilkan barisan bilangan ganjil sesuai input.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa rekursi merupakan metode yang efektif untuk menyelesaikan berbagai permasalahan yang memiliki pola berulang, seperti menampilkan deret bilangan, menghitung nilai tertentu, dan mencari faktor bilangan. Dengan menggunakan fungsi rekursif, program dapat dibuat lebih sederhana dan terstruktur karena permasalahan dipecah menjadi bagian yang lebih kecil. Namun, penggunaan rekursi harus memperhatikan adanya base case agar tidak terjadi pemanggilan tak hingga serta mempertimbangkan efisiensi penggunaan memori. Secara keseluruhan, rekursi memberikan pemahaman yang lebih mendalam tentang cara kerja fungsi dan sangat berguna dalam pengembangan algoritma.